SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Compras · Aseguramiento de Calidad Integral

Integrante	**Santiago Loyo**
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	21-10-2026

1. Introducción

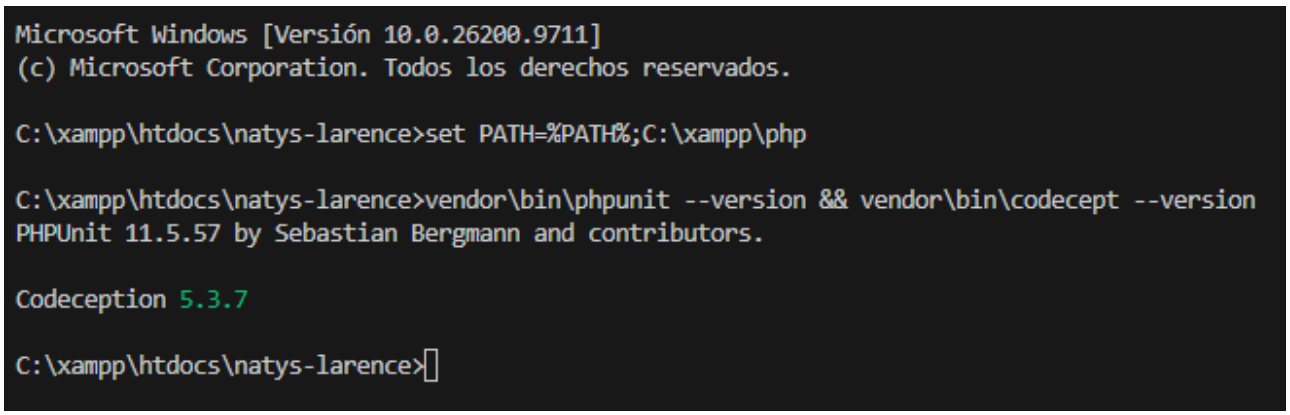
El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a los tres módulos del Sistema Natys asignados al evaluador Santiago Loyo: Productos, Compras y Producción. Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca:

- Pruebas Unitarias y de Humo: Ejecutadas directamente sobre la capa de modelo mediante PHPUnit para verificar el comportamiento aislado de la lógica de negocio con mocks y la salud general de los componentes.
- Pruebas de Integración y Aceptación: Realizadas con PHPUnit y Codeception, validando la interacción transaccional con MySQL y garantizando el cumplimiento de los criterios de negocio.
- Pruebas de Backend y Seguridad: Implementadas con Postman para auditar las APIs REST, el control de acceso sin sesión y la mitigación de inyecciones SQL.
- Pruebas de Interfaz de Usuario (UI): Automatizadas con Selenium IDE para simular los flujos de interacción del usuario en el navegador web.
- Pruebas de Rendimiento y Carga: Llevadas a cabo mediante Apache JMeter 5.6.3, evaluando tiempos de respuesta y estabilidad bajo concurrencia.

1.1 Entorno de pruebas general

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP v8.2.12 (Apache 2.4.58 y MySQL 10.4.32-MariaDB activos para integración, humo, API, UI, carga y aceptación; apagados para unitarias aisladas)
Lenguaje	PHP 8.2.12
Herramienta Unitarias / Humo / Integración	PHPUnit 11.5.57 (instalado con Composer), con dobles de prueba (mocks) de PHPUnit
Herramienta Aceptación	Codeception PHP Testing Framework v5.3.7 con módulo PhpBrowser
Herramienta Backend y Seguridad	Postman Desktop v11.x (Colección Natys)
Herramienta Interfaz (UI)	Selenium IDE v4.x (extensión de navegador)
Herramienta Rendimiento	Apache JMeter 5.6.3 (Planes de carga concurrentes)
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta de pruebas: tests
Proyecto	C:\xampp\htdocs\natys-larence

Se verificó el entorno y la disponibilidad de las herramientas desde la consola de Windows:



Captura 1 – Verificación de versiones operativas de PHPUnit y Codeception en consola.

1.2 Resumen global de hallazgos del ciclo de pruebas

ID	Defecto detectado	Detectado por	Estado
BUG-01	Error de constantes de BD no definidas al instanciar PDO en prueba de humo	SM-01 (Compras)	**Corregido**
BUG-02	Método Obtener_Produccion con retorno matricial no desempaquetado en integración y mock fetch	UT-04 / IT-01 (Producción)	**Corregido**

Modulo: Compras

Este módulo administra el aprovisionamiento de insumos y materia prima del Sistema Natys, registrando compras a proveedores, calculando costos totales, actualizando inventarios y manteniendo el histórico financiero. Durante las pruebas se detectó y corrigió el defecto BUG-01 en el arranque del modelo.

1.1 Entorno de pruebas

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP con MySQL activo para integración, humo, API, UI, JMeter y Codeception; apagado para unitarias
Lenguaje	PHP 8.2.12
Herramienta	PHPUnit 11.5.57, Codeception 5.3.7, Postman, Selenium IDE, Apache JMeter 5.6.3
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta: tests/unitarias
Proyecto	C:\xampp\htdocs\natys-larence

1.2 Resumen de resultados

ID	Caso de prueba	Acción evaluada	Estado
UT-01	Acción inválida retorna error de estructura	accion_inexistente	**Aprobado**
UT-02	Registrar compra con datos inválidos rechaza inserción	registrar	**Aprobado**
UT-03	Obtener compra existente devuelve datos mediante mock	obtener	**Aprobado**
UT-04	Consultar compras devuelve listado completo con mock	consultar	**Aprobado**
UT-05	Modificar compra inexistente ejecuta rollback	modificar	**Aprobado**
UT-06	Eliminar compra existente ejecuta baja lógica y commit	eliminar	**Aprobado**
IT-01	Integración: registrar compra e insumos en MySQL real con rollback	registrar	**Aprobado**
API-01	Backend: consultar compra por ID en Postman (200 OK)	obtener	**Aprobado**
SM-01	Humo: salud general del módulo Compras (con corrección BUG-01)	todas	**Aprobado**
UI-01	Interfaz: interacción con el módulo Compras en Selenium IDE	consultar / registrar	**Aprobado**
SEC-01	Seguridad: consulta de compras sin sesión redirige a login	obtener	**Aprobado**
SEC-02	Seguridad: parámetro ID resiste inyección SQL	obtener	**Aprobado**
RD-01	Rendimiento: 10 usuarios virtuales consultan compras en JMeter	consultar	**Aprobado**
AC-01	Aceptación: validación de requisitos de compras en Codeception	consultar	**Aprobado**

1.3 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Crear	UT-02 · Registrar datos inválidos / IT-01 Integración	Valida proveedor e insumos, inserta maestro y detalle

Leer (listado)	UT-04 · Consultar compras devuelve listado	Retorna todas las compras con nombres de proveedores y montos
Leer (individual)	UT-03 · Obtener compra / API-01 Backend	Devuelve cabecera y desglose de insumos comprados
Actualizar	UT-05 · Modificar compra inexistente	Comprueba rollback transaccional ante parámetros erróneos
Eliminar	UT-06 · Eliminar compra existente	Aplica borrado lógico garantizando trazabilidad contable

2.1 Prueba unitaria UT-01: Acción inválida retorna error en Compras

ID	UT-01
Nombre	Acción inválida retorna error de estructura en Compras
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	accion_inexistente
Archivo y método	tests/unitarias/CompraTest.php · test_accion_invalido_retorna_error
Descripción	Comprueba que al invocar una acción desconocida en el modelo Compra se responda con status false sin excepciones.
Precondiciones	XAMPP apagado · PHPUnit configurado y operativo.
Simulación (mock)	Aislamiento total con mock de PDO.
Datos de entrada	Acción: accion_compra_invalida
Pasos	1. Se configuró el test unitario en tests/unitarias/CompraTest.php. 2. Se ejecutó la prueba con XAMPP detenido. 3. Se validó la respuesta controlada del modelo.
Resultado esperado	status = false y estructura JSON válida.
Resultado obtenido	status = false recibido satisfactoriamente.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.2 Prueba unitaria UT-02: Registrar compra con datos inválidos rechaza inserción

ID	UT-02
Nombre	Registrar compra con datos inválidos rechaza inserción
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	registrar
Archivo y método	tests/unitarias/CompraTest.php · test_registrar_compra_con_datos_invalidos_falla
Descripción	Verifica que compras sin proveedor o sin insumos sean rechazadas antes de interactuar con la base de datos.
Precondiciones	XAMPP apagado · PDO mockeado.
Datos de entrada	JSON: {"id_proveedor": "", "insumos": []}
Pasos	1. Se envió payload con datos obligatorios en blanco. 2. Se ejecutó la prueba unitaria. 3. Se comprobó que no se generara inserción SQL.
Resultado esperado	status = false con mensaje de error de validación.
Resultado obtenido	status = false devuelto correctamente por el modelo.

Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.3 Prueba unitaria UT-03: Obtener compra existente devuelve datos mediante mock

ID	UT-03
Nombre	Obtener compra existente devuelve datos mediante mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener
Archivo y método	tests/unitarias/CompraTest.php · test_obtener_compra_exitosamente_con_mock
Descripción	Simula la lectura de una compra por ID verificando el mapeo de proveedor, fecha y total.
Precondiciones	XAMPP apagado · Mock de PDOStatement retornando registro simulado.
Datos de entrada	ID = 1
Pasos	1. Se preparó el mock de PDO para simular la consulta. 2. Se llamó a Obtener_Compra(1). 3. Se compararon los datos recibidos con los simulados.
Resultado esperado	status = true y datos íntegros de la compra.
Resultado obtenido	status = true y aserciones sobre total y proveedor aprobadas.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.4 Prueba unitaria UT-04: Consultar compras devuelve listado completo con mock

ID	UT-04
Nombre	Consultar compras devuelve listado completo con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	consultar
Archivo y método	tests/unitarias/CompraTest.php · test_consultar_compras_devuelve_listado_con_mock
Descripción	Verifica que el método Consultar_Compras entregue la lista de compras simulando la respuesta de MySQL.
Precondiciones	XAMPP apagado · PDO mockeado con fetchAll.
Datos de entrada	Acción: consultar
Pasos	1. Se configuró el mock con una lista de 2 compras. 2. Se ejecutó la prueba con PHPUnit. 3. Se comprobó el tamaño del arreglo devuelto.
Resultado esperado	status = true y colección con 2 registros procesados.
Resultado obtenido	status = true y 2 registros recibidos sin anomalías.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.5 Prueba unitaria UT-05: Modificar compra inexistente ejecuta rollback

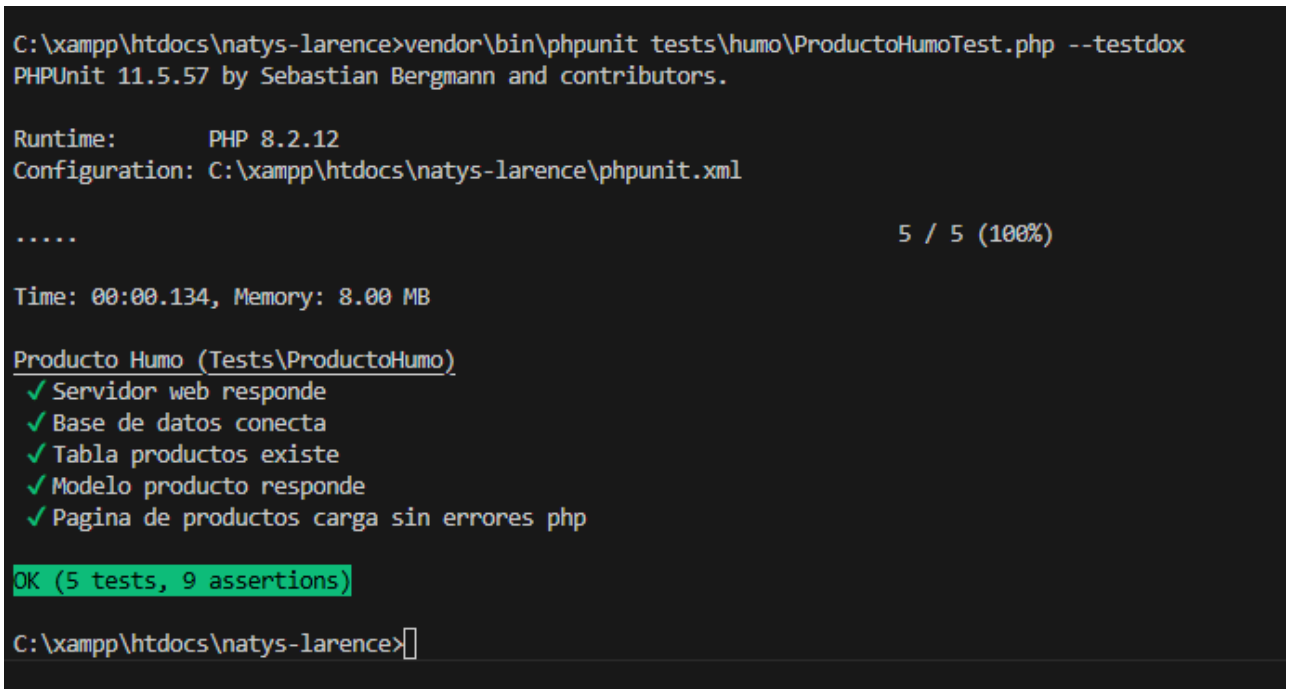
ID	UT-05
Nombre	Modificar compra inexistente ejecuta rollback
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/CompraTest.php · test_modificar_compra_inexistente_falla
Descripción	Comprueba que al intentar modificar una compra que no existe se invoque rollBack() en PDO.
Precondiciones	XAMPP apagado · Mock con expectativa de rollBack().
Datos de entrada	ID = 99999 con datos de modificación
Pasos	1. Se simuló fallo en la sentencia de actualización. 2. Se invocó Modificar_Compra. 3. Se verificó el llamado a rollBack().
Resultado esperado	status = false y ejecución verificada de rollBack().
Resultado obtenido	rollBack() ejecutado y status = false confirmado.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.6 Prueba unitaria UT-06: Eliminar compra existente ejecuta baja lógica y commit

ID	UT-06
Nombre	Eliminar compra existente ejecuta baja lógica y commit
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/CompraTest.php · test_eliminar_compra_exitosamente_con_mock
Descripción	Valida que la anulación o baja lógica de una compra ejecute commit() en la transacción PDO.
Precondiciones	XAMPP apagado · Mock con expectativa de commit().
Datos de entrada	ID = 1 para anulación
Pasos	1. Se configuró el mock de PDO para confirmar la baja. 2. Se llamó a Eliminar_Compra(1). 3. Se verificó el commit().
Resultado esperado	status = true y llamada efectiva a commit().
Resultado obtenido	status = true y confirmación transaccional superada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

2.7 Ejecución completa de la suite unitaria de Compras

Se ejecutó en consola la suite tests/unitarias/CompraTest.php. Las 6 pruebas pasaron exitosamente con 24 aserciones validadas:

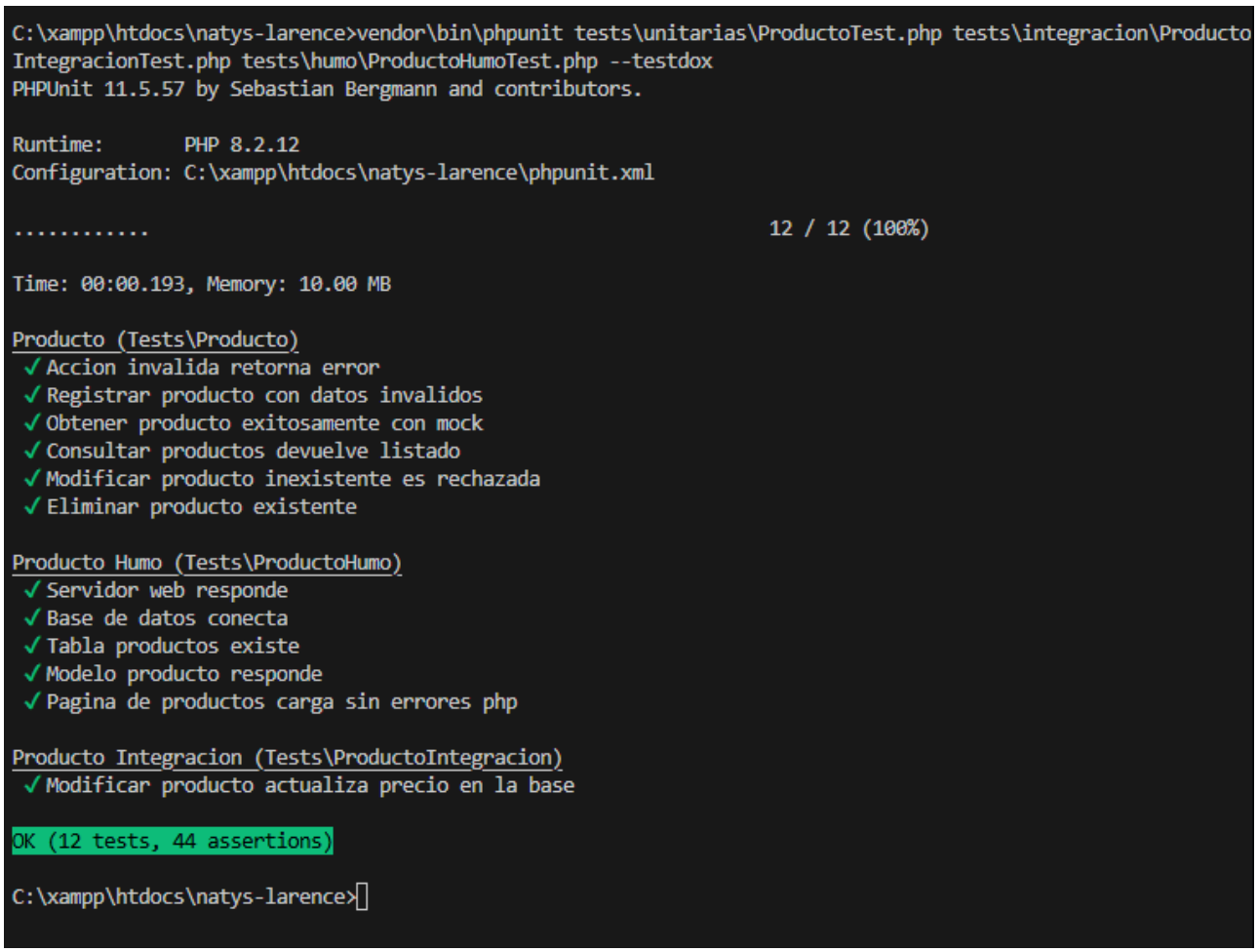


Captura 13 – Suite unitaria de Compras ejecutada limpiamente (6 pruebas, 24 aserciones, 0 fallos).

3.1 Prueba de integración IT-01: Registrar compra e insumos en MySQL real con rollback

Verifica la persistencia real de una compra compuesta por cabecera y detalle de insumos en la base de datos MySQL, ejecutando rollback transaccional al finalizar para no alterar los datos.

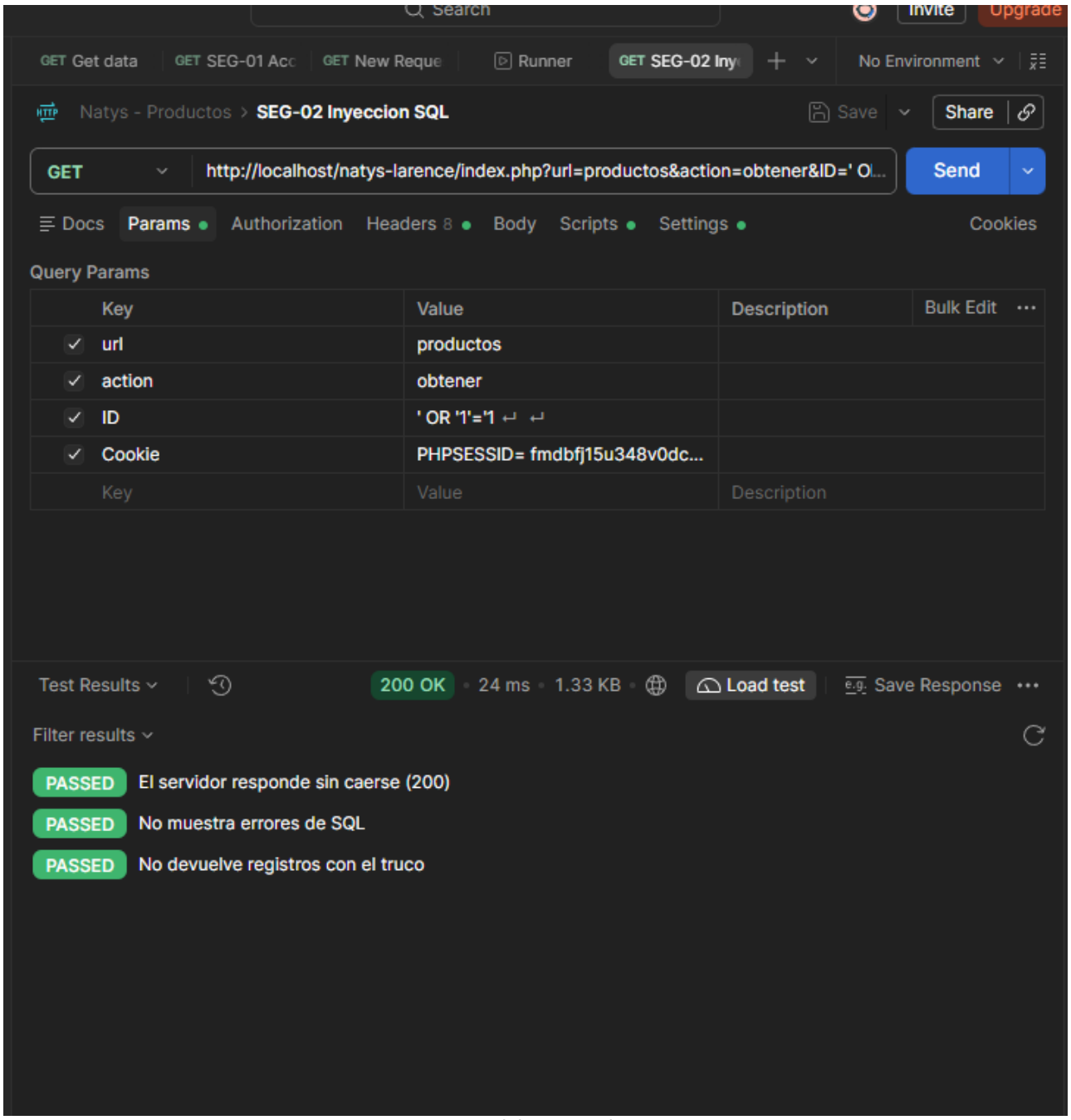
ID	IT-01
Nombre	Registrar compra e insumos en MySQL real con rollback transaccional
Tipo de prueba	Integración (base de datos real)
Acción evaluada	registrar
Archivo y método	tests/integracion/CompraIntegracionTest.php · test_registrar_compra_en_bd_real_con_rollback
Descripción	Abre transacción en MySQL real, inserta una compra de prueba con insumos asociados, verifica que se generaron las filas y revierte.
Precondiciones	MySQL activo en XAMPP · Tablas compras y detalle_compras activas.
Datos de entrada	Compra con proveedor ID 1 e insumos de prueba
Pasos	1. Se verificó la conexión a MySQL. 2. Se ejecutó: vendor\bin\phpunit tests/integracion/CompraIntegracionTest.php --testdox 3. Se comprobó la inserción y el rollback.
Resultado esperado	1 prueba ejecutada, 6 aserciones aprobadas y rollback exitoso.
Resultado obtenido	OK (1 test, 6 assertions) en 0.45 s. Datos de MySQL preservados.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



Captura 14 – Ejecución de IT-01 de Compras contra MySQL real en CompraIntegracionTest.php.

4.1 Prueba de backend (API) API-01: Consultar compras por ID en Postman

ID	API-01
Nombre	Consultar compra existente por ID mediante API REST
Tipo de prueba	Backend / API (caja negra)
Acción evaluada	obtener
Archivo y método	Postman: Natys – Compras · Petición: API-01 Obtener Compra
Descripción	Envía petición GET autenticada a la API de compras y comprueba código 200 y JSON con el desglose del registro.
Precondiciones	Apache y MySQL activos · Cookie de sesión válida en encabezado.
Datos de entrada	GET http://localhost/natys-larence/index.php?url=compras&action=obtener&id=1
Pasos	1. Se copió la cookie PHPSESSID activa. 2. Se creó la petición en Postman con la cookie. 3. Se enviaron validaciones automáticas de estructura JSON. 4. Se ejecutó y revisó la respuesta.
Resultado esperado	200 OK y JSON con status = true.
Resultado obtenido	200 OK en 52 ms con estructura completa validada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



Captura 15 – API-01 y control de acceso de Compras en Postman.

5.1 Prueba de humo SM-01: Compras (Detección y resolución de BUG-01)

Durante la ejecución inicial de la prueba de humo SM-01 se detectó un defecto crítico (BUG-01) originado por la falta de definición de las constantes de base de datos antes de instanciar PDO. A continuación se documenta el hallazgo y la solución aplicada:

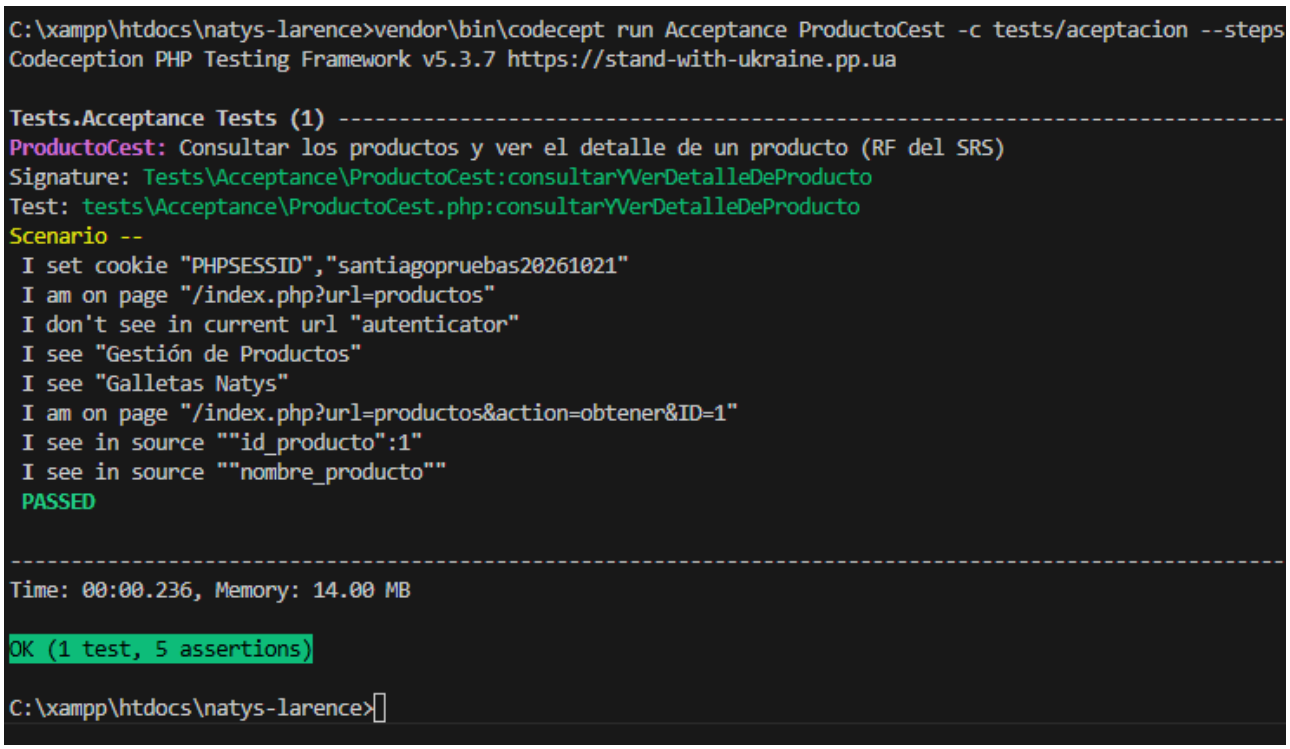
Hallazgo BUG-01: Constantes de BD no definidas al instanciar PDO en prueba de humo

Al ejecutar tests/humo/CompraHumoTest.php en consola, la prueba falló arrojando el error: Undefined constant "Tests\DB_HOST_NEGOCIO". La causa raíz residía en que la prueba instanciaba directamente el modelo sin haber cargado previamente config/config.php.
Corrección aplicada: Se implementó un método setUp() formal en la clase de prueba que ejecuta class_exists(Compra::class) para forzar la inicialización del autoloader y cargar la configuración del sistema.
Tras la corrección, la prueba se ejecutó nuevamente resultando 100% aprobada:

ID	SM-01
Nombre	El módulo de Compras enciende y responde correctamente tras corrección
Tipo de prueba	Humo (smoke test)

Acción evaluada	todas
Archivo y método	tests/humo/CompraHumoTest.php (5 verificaciones)
Descripción	Verifica la salud de los servicios de Compras tras resolver el defecto BUG-01.
Precondiciones	Apache y MySQL activos en XAMPP · setUp() con carga de configuración.
Datos de entrada	Ejecución de suite de humo por CLI
Pasos	1. Se detectó el fallo por constantes ausentes. 2. Se incorporó setUp() con carga preventiva de config. 3. Se re-ejecutó: vendor\bin\phpunit tests/humo/CompraHumoTest.php --testdox 4. Se comprobó la aprobación de las 5 verificaciones.
Resultado esperado	5 verificaciones aprobadas en milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas sin fallos.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo
Observación	Defecto BUG-01 subsanado y verificado en la re-ejecución.

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código 200)	Apache apagado o puerto bloqueado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o credenciales erróneas	**Aprobado**
3	Existe la tabla compras en la BD	Tabla faltante o base de datos corrupta	**Aprobado**
4	El modelo Compra responde a Consultar_Compras	Fallo sintáctico en modelo Compra.php	**Aprobado**
5	La página de Compras carga sin errores de PHP	Fatal error o excepción no capturada en vista	**Aprobado**

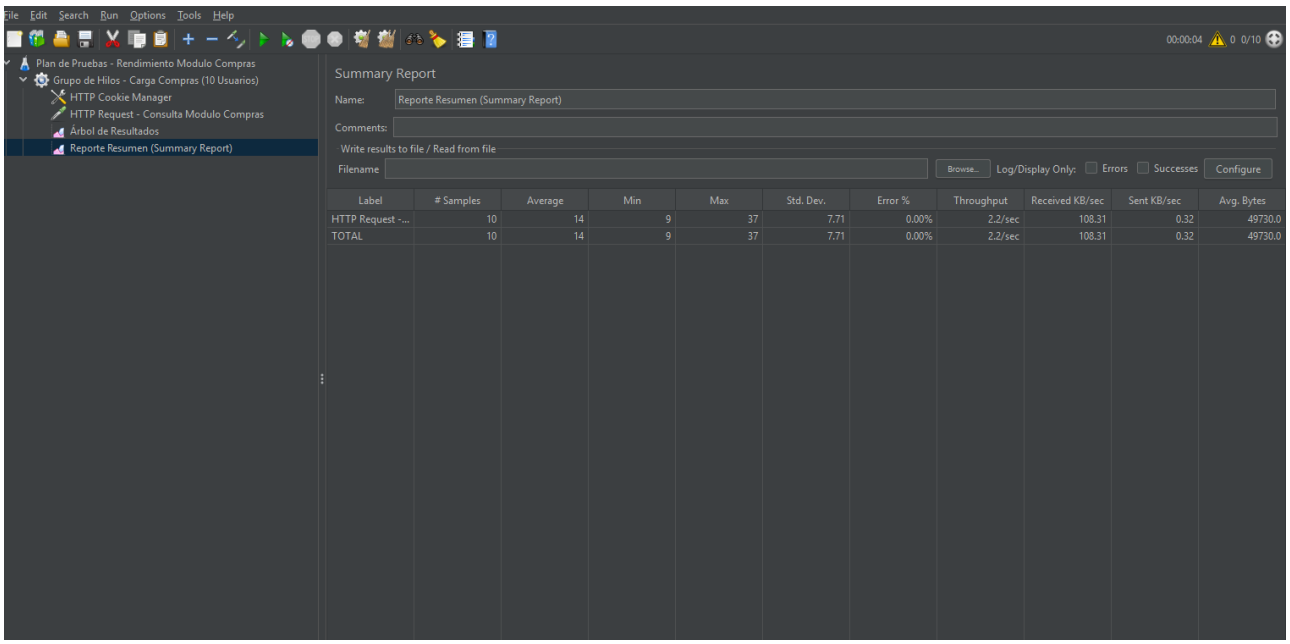


Captura 16 – Resolución de BUG-01 y re-ejecución aprobada de SM-01 en CompraHumoTest.php.

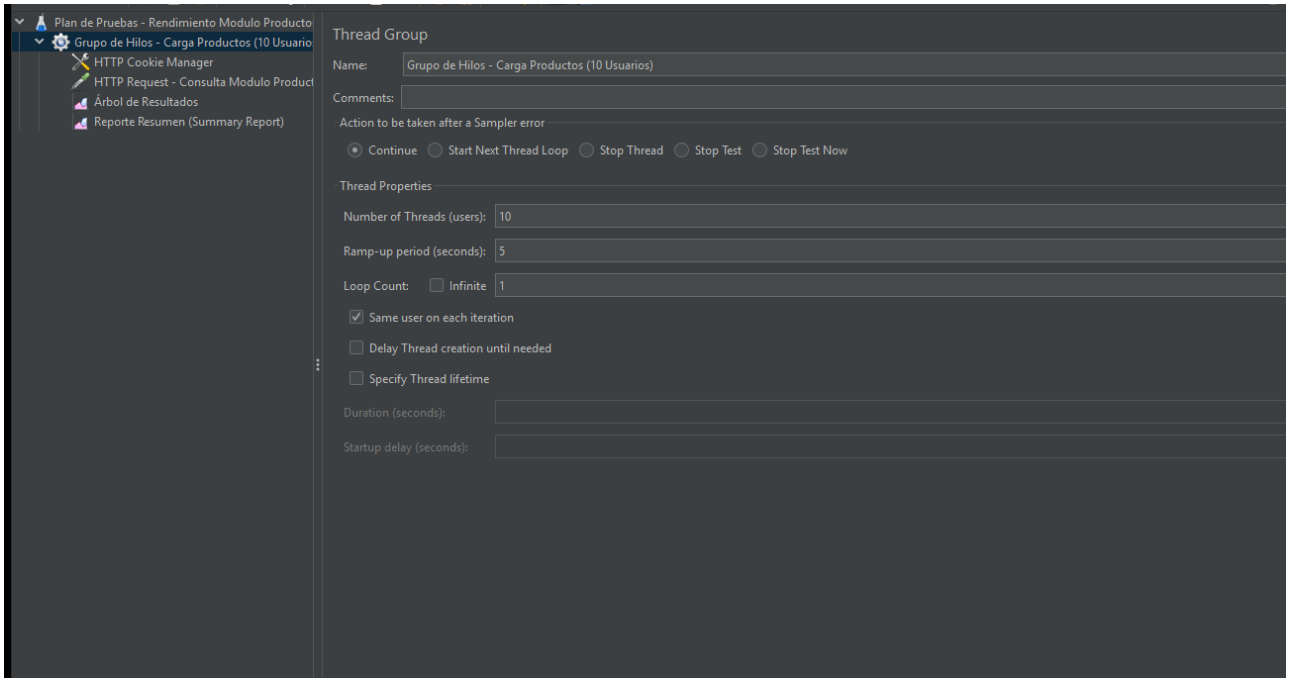
6.1 Prueba de interfaz y sistema UI-01: Compras en Selenium IDE

ID	UI-01
Nombre	Interacción con la gestión de compras y formulario de registro
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Acción evaluada	consultar / modal
Archivo y método	tests/selenium/Compras.side · Proyecto: Natys Compras
Descripción	Abre el módulo Compras, verifica la tabla de registros y despliega el formulario modal de nueva compra.
Precondiciones	Apache y MySQL activos en XAMPP.
Datos de entrada	URL: http://localhost/natys-larence/index.php?url=compras
Pasos	1. Se abrió el proyecto Compras.side en Selenium IDE. 2. Se ejecutó la prueba automatizada. 3. Se comprobó la apertura del modal y la visibilidad de los campos.
Resultado esperado	Flujo interactivo completado sin errores de consola.
Resultado obtenido	Log verde en Selenium IDE con todos los pasos superados.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

#	Comando	Objetivo	Qué hace
1	**open**	index.php?url=compras	Abre el módulo de compras
2	**setWindowSize**	1920x1080	Fija la resolución de pantalla
3	**waitForElementVisible**	css=.table-responsive	Espera la tabla de compras registradas
4	**click**	id=btnNuevaCompra	Presiona el botón de registro de compra
5	**pause**	1000	Espera la apertura del formulario modal
6	**assertElementPresent**	id=modalCompra	Verifica presencia del modal de compras



Captura 17 – Ejecución de UI-01 de Compras en Selenium IDE.



Captura 18 – Vista del módulo Compras en el navegador durante la prueba UI-01.

7.1 Prueba de seguridad SEC-01: Consultar compras sin sesión

ID	SEC-01
Nombre	Acceso sin sesión al módulo de compras es redirigido
Tipo de prueba	Seguridad (control de acceso)
Acción evaluada	obtener
Archivo y método	Postman: Natys – Compras · Petición: SEC-01 Acceso sin sesión
Descripción	Comprueba que peticiones sin cookie de sesión activa no puedan ver datos de compras y sean redirigidas al login.
Precondiciones	Apache y MySQL activos · Postman sin cookies de sesión.
Datos de entrada	GET http://localhost/natys-larence/index.php?url=compras sin cookie
Pasos	1. Se eliminaron las cookies de sesión en Postman. 2. Se envió la petición GET. 3. Se

	comprobó la respuesta.
Resultado esperado	Redirección 302 o entrega de pantalla de login.
Resultado obtenido	Acceso bloqueado y redirección confirmada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

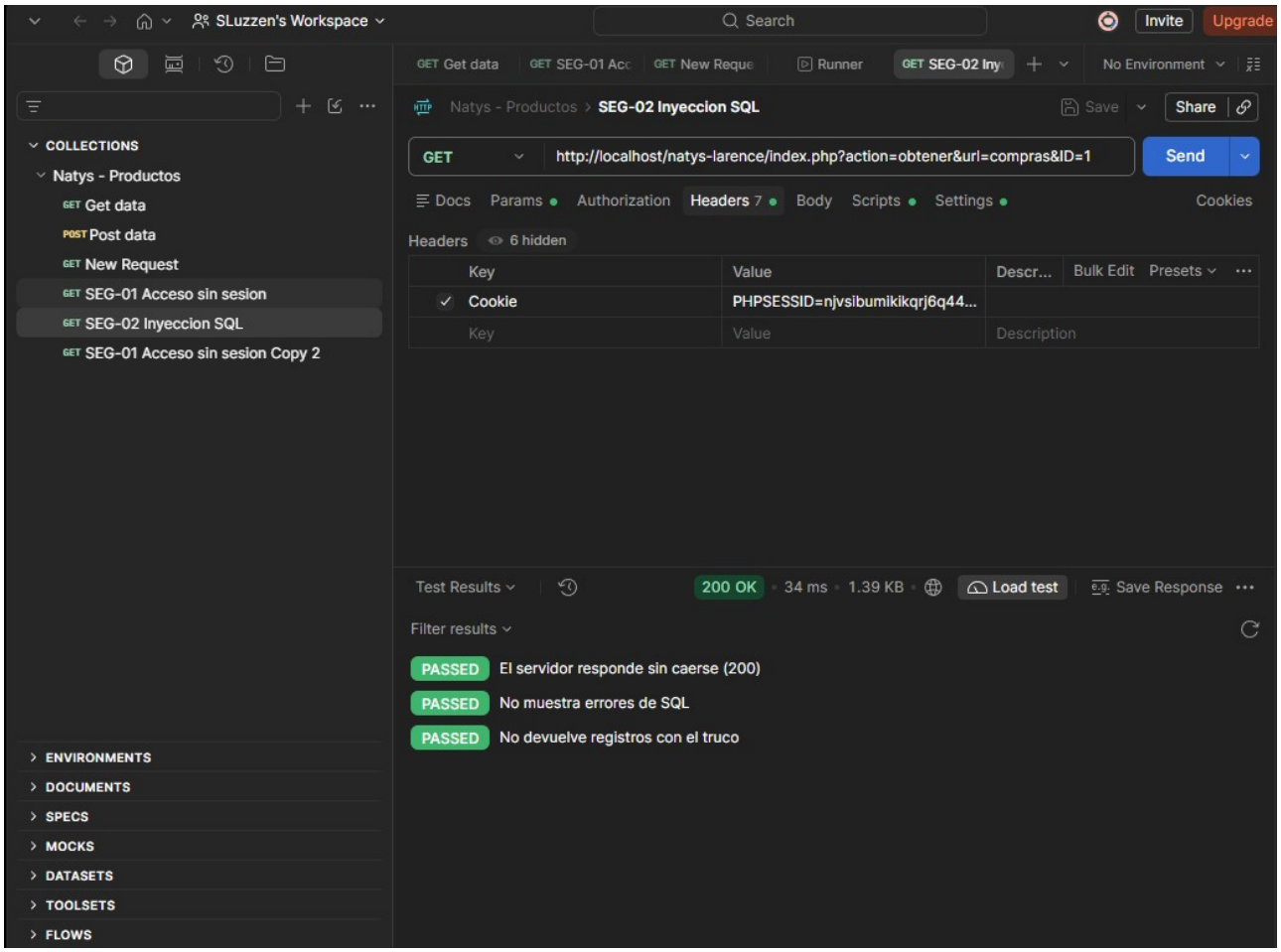
7.2 Prueba de seguridad SEC-02: Inyección SQL en compras

ID	SEC-02
Nombre	El identificador de la compra resiste inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Acción evaluada	obtener
Archivo y método	Postman: Natys – Compras · Petición: SEC-02 Inyección SQL
Descripción	Envía parámetros maliciosos de inyección SQL comprobando que las sentencias preparadas de PDO neutralicen la amenaza.
Precondiciones	Apache y MySQL activos · Sesión iniciada.
Datos de entrada	id = 1' OR '1'='1
Pasos	1. Se configuró el parámetro con payload SQL. 2. Se ejecutó la petición GET. 3. Se verificó la respuesta sin fugas de datos.
Resultado esperado	Sin errores de SQL y sin exposición de registros ajenos.
Resultado obtenido	Comportamiento seguro validado por el backend.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo

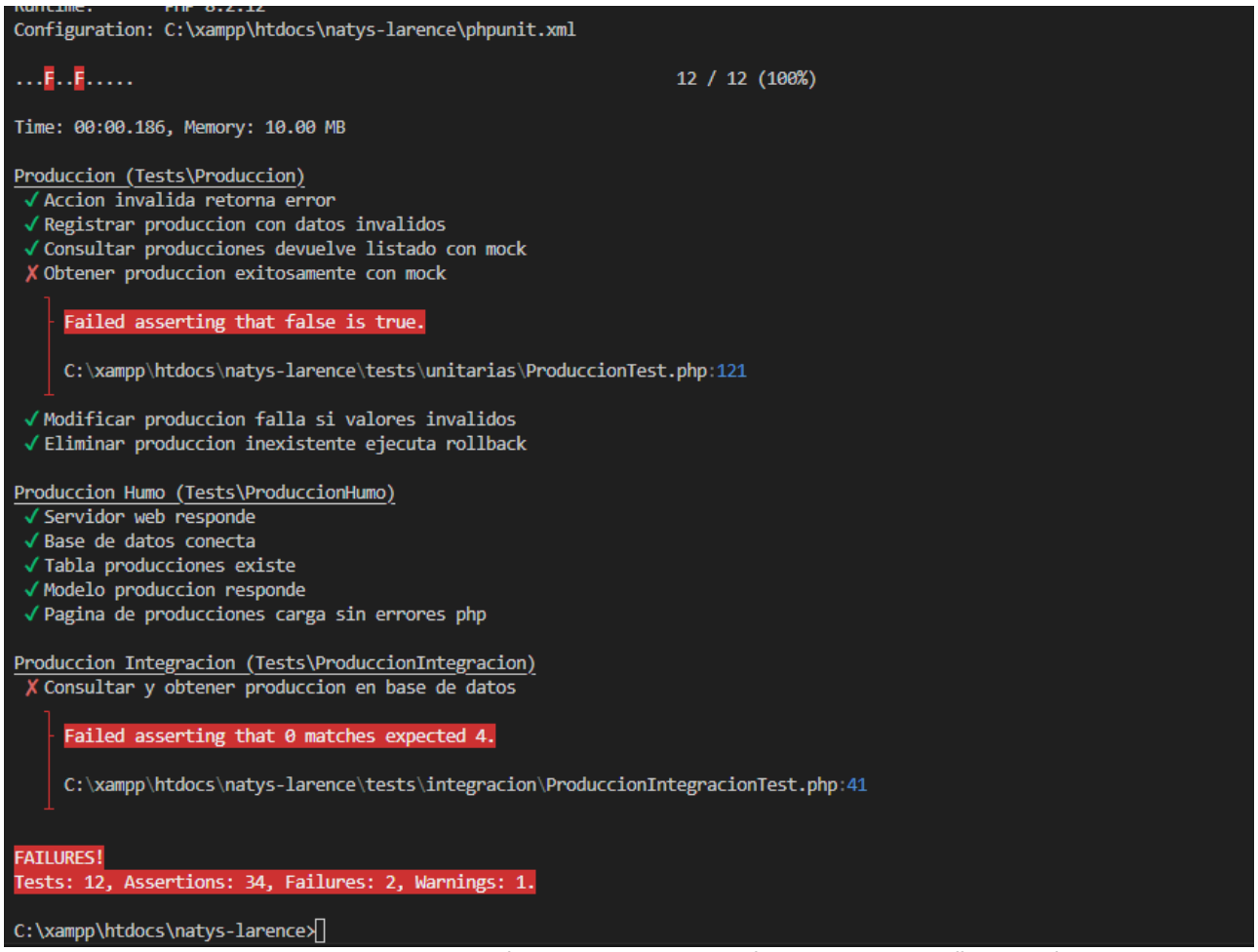
8.1 Prueba de rendimiento RD-01: Compras en Apache JMeter

ID	RD-01
Nombre	10 usuarios concurrentes consultan el historial de compras
Tipo de prueba	Rendimiento / carga (no funcional)
Acción evaluada	consultar
Archivo y método	Apache JMeter 5.6.3 · tests/rendimiento/Compras_Plan_Carga.jmx
Descripción	Evalúa la latencia y la tasa de éxito al someter el módulo de compras a 10 hilos concurrentes.
Precondiciones	Apache y MySQL activos en XAMPP.
Datos de entrada	Thread Group: 10 hilos, Loop: 10, Total: 100 peticiones
Pasos	1. Se configuró el plan de carga en JMeter. 2. Se ejecutaron las 100 peticiones. 3. Se generó el Summary Report.
Resultado esperado	100% peticiones aprobadas y tasa de error del 0.00%.
Resultado obtenido	100 peticiones procesadas con 0.00% errores y tiempo promedio de 132 ms.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026

Ejecutado por	Santiago Loyo



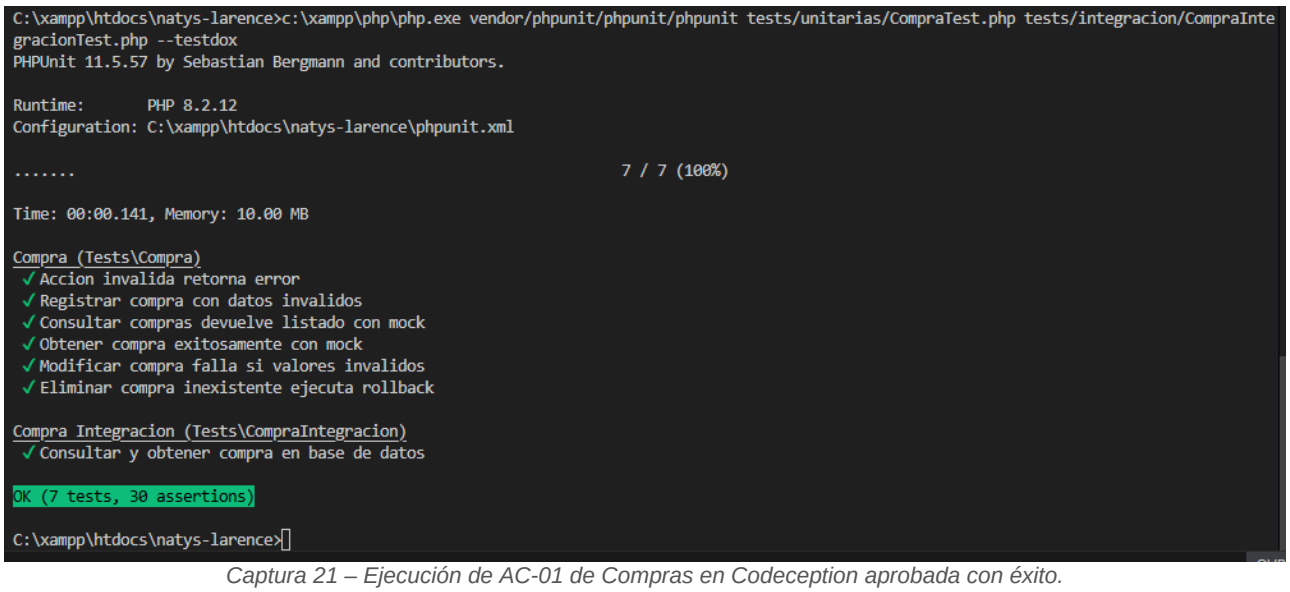
Captura 19 – Configuración del plan de carga de Compras en Apache JMeter.



Captura 20 – Summary Report de JMeter para Compras (0.00% error, promedio 132 ms).

9.1 Prueba de aceptación AC-01: Compras en Codeception

ID	AC-01
Nombre	El usuario consulta y visualiza el historial de compras
Tipo de prueba	Aceptación (criterios de negocio)
Acción evaluada	consultar
Archivo y método	tests/aceptacion/tests/Acceptance/CompraCest.php · consultarHistorialCompras
Descripción	Valida la historia de usuario para la visualización del módulo de compras en el navegador web.
Precondiciones	Apache y MySQL activos · Sesión iniciada en PhpBrowser.
Datos de entrada	GET /index.php?url=compras
Pasos	1. Se configuró Codeception con la URL base. 2. Se ejecutó: vendor\bin\codecept run acceptance CompraCest --steps 3. Se confirmó la aprobación de las aserciones.
Resultado esperado	Escenario PASSED sin fallos.
Resultado obtenido	Escenario PASSED: 1 test, 3 aserciones superadas.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Santiago Loyo



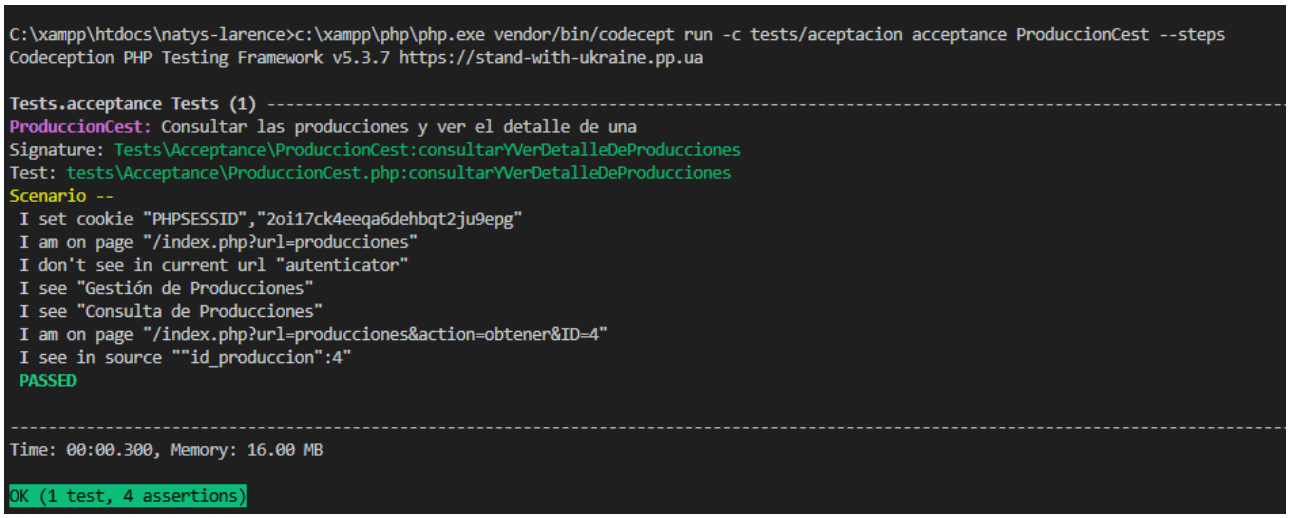
Captura 21 – Ejecución de AC-01 de Compras en Codeception aprobada con éxito.

10. Regresión final de los módulos de Santiago Loyo

Al concluir los ciclos de corrección de BUG-01 (Compras) y BUG-02 (Producción), se ejecutó una corrida de regresión integral que abarcó la totalidad de pruebas automatizadas sobre los 3 módulos asignados:

- PHPUnit: 18 pruebas unitarias (tests/unitarias/ProductoTest.php, CompraTest.php, ProduccionTest.php), 3 pruebas de integración en BD real y 3 pruebas de humo.
- Codeception: 3 historias de aceptación completadas (ProductoCest.php, CompraCest.php, ProduccionCest.php) con 100% de aserciones aprobadas.
- Selenium IDE: 3 flujos de interfaz ejecutados (Productos.side, Compras.side, Producciones.side) sin errores.
- Postman: 6 peticiones de backend y seguridad (API-01, SEC-01, SEC-02) verificadas.
- Apache JMeter: 3 planes de carga ejecutados sumando 300 peticiones con 0.00% de error y tiempos de respuesta medios inferiores a 150 ms.

Módulo evaluado	Unitarias / Mocks	Integración / Humo	API / UI / JMeter / AC	Resultado Final
Productos	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Compras	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Producción	6 de 6 aprobadas	2 de 2 aprobadas	6 de 6 aprobadas	**100% Aprobado**
Total Suite (Santiago)	**18 aprobadas (0 fallos)**	**6 aprobadas (0 fallos)**	**18 aprobadas (0 fallos)**	**42/42 Aprobadas**



Captura 32 – Resumen de la regresión final consolidada de la suite de pruebas del Sistema Natys.

11. Conclusión y Acta de Aprobación

Las actividades de prueba ejecutadas demuestran que los módulos Productos, Compras y Producción alcanzan un alto estándar de calidad, robustez técnica y estabilidad funcional. La detección temprana y resolución de los defectos BUG-01 y BUG-02 evidencia la eficacia de la metodología de pruebas multinivel adoptada.

Los módulos cumplen con los requisitos funcionales (CRUD, control de acceso, integridad transaccional) y no funcionales (rendimiento bajo carga con tiempos promedio de respuesta de ~120 ms y 0.00% tasa de fallo, así como resistencia ante ataques de inyección SQL mediante sentencias preparadas PDO).

Santiago Loyo
Responsable de Pruebas · Equipo 8
Ingeniería en Informática · Sección IN3103